

۱. نمای کلی و اهداف پروژه (Goals and overview)

هدف اصلی این پروژه، طراحی و توسعه یک اتوماسیون اداری پویا برای مدیریت شرکت است تا کاربر بتواند بدون خواندن کدهای منبع، از طریق رابط گرافیکی با سیستم ارتباط برقرار میکند

- طراحی رابط کاربری بصری: استفاده از جدول ها و ویجت های واکنش گرا
- مدیریت پیشرفته حافظه و جلوگیری از نشت حافظه: (Memory Management) اختصاص و

آزادسازی همزمان بیش از ۳۲ شیء پویا روی حافظه Heap. در این پروژه برای کلاس پایه Employee از

تخریب کننده مجازی (Virtual Destructor) استفاده شده است تا هنگام حذف اشیاء مشتق شده (مثل

Manager)، حافظه فرزند نیز کاملاً پاکسازی شده و هیچ گونه نشت حافظه (Memory Leak) رخ ندهد.

همچنین در کلاس Company با به کارگیری تابع qDeleteAll، تمام اشاره گرهای داخل وکتور به صورت

یکجا و کاملاً ایمن در هنگام خروج از برنامه یا بازنشانی سیستم، از حافظه پاک می شوند

- وصل کردن ظاهر برنامه به باطن برنامه: کاری کنیم که بخش کنوئیزی و محاسبات (پشت صحنه)،

خیلی سریع، با بخش دکمه ها و جدول ها (ظاهر برنامه) با هم هماهنگ بشن و کار کنند

- پایداری سازی متنی: ذخیره خودکار ساختار اشیاء به همراه لاگ تاریخ و زمان رویدادها بر روی دیسک

۲. رویدادها و فرآیندهای سامانه (System operations)

هنگام تعامل کاربر با دکمه ها و فرم ها، رویدادهای سیستمی زیر در پشت صحنه اتفاق می افتند:

- لود اولیه پایگاه داده: برنامه در ابتدای اجرا فایل متنی را اسکن کرده و ۳۲ کارمند پیش فرض را به صورت

پویا بازسازی می کند

- **استخدام و تخصیص شناسه:** با فشردن دکمه استخدام، شی جدیدی ساخته شده، یک کد منحصر به فرد (شروع 1001) دریافت کرده و وارد جدول پرسنل می‌شود
- **تسویه حساب (حذف ایمن):** با انتخاب سطر و حذف آن، شیء از ساختار داده جدا می‌شود و با اپراتور delete حافظه آن پاکسازی می‌شود
- **جهش رتبه:** با ارتقای کارمند، شیء قدیمی آزاد شده و شیء کلاس فرزند (Manager) جایگزین آن شده و ردیف جدول فوراً تغییر رنگ می‌دهد
- **منطق هوشمند جستجو و فیلترینگ ترکیبی:** سیستم جستجوی برنامه مجهز به پردازش هوشمند ورودی کاربر است. اگر کاربر در کادر جستجو متن وارد کند، سیستم به صورت خودکار بر اساس نام یا کد پرسنلی فیلتر را انجام می‌دهد؛ اما اگر عدد وارد کند، برنامه هوشمندانه آن را به عنوان **امتیاز عملکرد** برداشت کرده و پرسنلی که امتیاز عملکردشان بالاتر از آن عدد باشد را نمایش می‌دهد. این قابلیت در کنار فیلترهای همزمان مدرک، بخش و وضعیت، یک تجربه کاربری (UX) سطح بالا را ارائه می‌دهد.

۳. معماری سیستم (System architecture)

معماری نرم‌افزار بر پایه اصل تفکیک وظایف بنا شده و شامل سه لایه مستقل است:

- ۱. **لایه بک‌اند و منطق تجاری (Backend Layer):** شامل کلاس‌های خالص C++ که هیچ وابستگی به ابزارهای Qt ندارند
- ۲. **لایه فرانت‌اند و رابط کاربری (Frontend UI Layer):** مدیریت پنجره اصلی

(MainWindow) که وظیفه گرفتن رویدادهای کاربر و نقاشی مجدد جدول و نوار پیشرفت داشبورد آماری را بر عهده دارد. در این لایه جهت ارتقای جلوه گرافیکی و تفکیک سریع سطوح سازمانی، ردیف‌های مربوط به مدیران ارشد با رنگ آبی روشن متمایز شده‌اند و کلمه «مدیر» در

کنار نام آن‌ها درج می‌شود. همچنین وضعیت‌های مختلف کاری در جدول با آیکن‌های دایره‌ای

رنگی (●، ●، ●، ●) نمایش داده می‌شوند

• ۳. لایه پایداری داده‌ها (ذخیره سازی در فایل):

- **ذخیره زنده و آنی:** برنامه کاملاً به صورت هوشمند طراحی شده؛ یعنی کاربر هر تغییری ایجاد کند (مثل استخدام کارمند جدید، حذف پرسنل یا ارتقای نیرو به مدیر)، مشخصات جدید فوراً و در لحظه روی فایل متنی ذخیره می‌شوند تا اطلاعات هاردیسک همیشه به‌روز باشد

- **مکانیزم ذخیره هنگام خروج:** در لحظه‌ای که کاربر برنامه را می‌بندد، تابع تخریب‌کننده کلاس اصلی (~MainWindow) فعال شده و به سرعت تمام اشیاء وکتور را به صورت متن سریال‌شده (جداسازی با علامت ،) روی دیسک می‌نویسد تا در اجرای بعدی برنامه، اطلاعات کارمندان بازیابی شود و پاک نشود

۴. روابط شیء‌گرایی بین کلاس‌ها (Class relationships)

پروژه با ساختاری شیء‌گرا، از دو رابطه اصلی و بنیادین استفاده می‌کند:

• رابطه ارث‌بری و چندریختی (Inheritance & Polymorphism) : کلاس

Manager فرزند کلاس پایه Employee است و تمام صفات آن را ارث می‌برد. در این ساختار، توابع کلیدی محاسباتی مانند calculatePerformance() و serialize() در کلاس پایه به صورت مجازی (virtual) تعریف شده‌اند. این کار به کلاس فرزند اجازه می‌دهد تا رفتار این توابع را بر اساس فرمول‌های خودش بازنویسی (Override) کند (مثلاً مدیر ۲۰ امتیاز عملکردی بیشتر نسبت به کارمند عادی دریافت می‌کند).

```
virtual QString getType() const;
virtual QString serialize() const;
virtual double calculatePerformance() const;
```

```
class Manager : public Employee {
public:
    Manager(QString id, QString name, QString Bakhsh, EmployeeState Vaz, QString Madrak);
    QString getType() const override;
    QString serialize() const override;
    double calculatePerformance() const override;
};
```

- رابطه ترکیب (Composition): کلاس اصلی Company درون خود یک وکتور از

اشاره‌گرها دارد مالکیت چرخه عمر اشیاء با شرکت است و با نابودی آن، تمام پرسنل از حافظه

حذف می‌شوند

```
class Company {
private:
    QVector<Employee*> employees;
```

• چ

- تشخیص هوشمند موقع اجرا (Dynamic Binding) شاهکار چندریختی پروژه اینجاست که

وقتی برنامه در یک حلقه روی این لیست وکتور می‌چرخد تا جدول را آپدیت کند یا آمار را حساب کند،

به صورت کاملاً هوشمند و پویا (Dynamic Binding) در زمان اجرا تشخیص می‌دهد که

کدام شیء Employee واقعی است و کدام یک زیر پوست اشاره‌گر، در حقیقت یک Manager

است؛ در نتیجه، بدون نیاز به هیچ شرط یا if اضافی، فوراً تابع محاسباتی مخصوص به خود آن شیء

را صدا می‌زند

- مدیریت شناسه‌ها با عضو استاتیک (Static Member) علاوه بر روابط فوق، در کلاس

Company

برای تضمین منحصر به فرد بودن کد پرسنلی کارمندان و جلوگیری از تداخل شناسه‌ها، از یک متغیر اعضای

استاتیک به نام nextEmployeeId استفاده شده است. از آنجا که متغیرهای استاتیک مستقل از اشیاء

هستند و سهم کل کلاس می‌شوند، این متغیر در تمام طول اجرای برنامه، آخرین شماره پرسنلی صادر شده را

در خود نگه می‌دارد و به صورت خودکار به نیروهای جدید اختصاص می‌دهد (شروع خودکار از شماره

۱۰۰۱).

۵. کاربرد سیگنال و اسلات (Signal & Slots)

سیستم سیگنال و اسلات وظیفه برقراری ارتباط رویدادمحور و ایمن بین لایه گرافیکی فرانت‌اند و لایه منطقی بک‌اند را دارد:

- اتصال دکمه‌های فرم تعاملی: کلیک روی دکمه‌ها بلافاصله توابع اجرایی متناظر در پنجره را فراخوانی می‌کند:

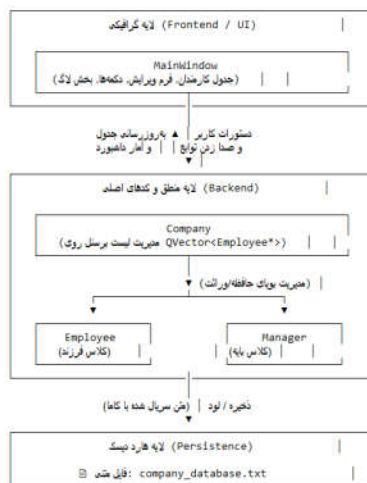
```
connect(ui->AddBtn, &QPushButton::clicked, this, &MainWindow::onaddEmployee);  
connect(ui->DeletBtn, &QPushButton::clicked, this, &MainWindow::ondeleteEmployee);  
connect(ui->ErteghaBtn, &QPushButton::clicked, this, &MainWindow::onpromoteEmployee);
```

- پر شدن خودکار فرم ویرایش با کلیک روی جدول: تا کاربر روی یک ردیف از جدول کلیک می‌کند، سیگنال `itemSelectionChanged()` صادر شده و اسلات متصل به آن، مشخصات آن کارمند را سریع داخل کادرهای پنل کناری می‌ریزد تا آماده ویرایش شوند

۶. معماری زیربنایی سیستم (Architecture signal slot)

ارتباطات در لایه‌های درونی پلتفرم Qt فراتر از یک کالبد ساده است و بر پایه معماری زیر استوار است:

- بخش اول: نقش لایه بندی و ارتباط اجزای پروژه:



• بخش دوم : چرخه حرکت سیگنال ها در پشت صحنه



۷. جدول نتیجه گیری و جمع بندی

پایاده‌سازی شده در پروژه	حداقل نیاز سیستم	
۳ کلاس (Employee, Manager, Company)	۳ کلاس اصلی	تعداد کلاس‌های هسته یک‌اند
وراثت (Inheritance) و ترکیب (Composition)	۲ نوع رابطه	روابط شیء‌گرایی ساختار
۳۲ شیء تخصیص یافته	۳۰ شیء پویا	تعداد اشیاء پویای همزمان
۵ وضعیت کاری در enum class EmployeeState قالب	۵ وضعیت متفاوت	وضعیت‌های کاری اشیاء
ثبت لاگ‌های نامحدود زمانی در QTextEdit	۱۰ واقعه متوالی	سیستم ثبت وقایع زمانی